Towards a Large Scale LOCARN Design
Low Opex & Capex Architecture for Resilient Networks

Damien Le Quéré*, Christophe Betoule*, Rémi Clavier*, Yassine Hadjadj-Aoul†,
Adlen Ksentini† and Gilles Thouénon*
*Orange Labs, Lannion, France
Email: firstname.lastname@orange.com
†IRISA, University of Rennes I, France
Email: firstname.lastname@irisa.fr

***Abstract*—We proposed recently LOCARN: an alternative network architecture providing a very simple packet connectivity layer, which is able to self-adapt its routing paths to both the effective traffic fluctuations and network resource changes. In this paper, we propose two design improvements in order to increase the number of communications supported within a domain. Then, we estimate by simulation the gain of the two proposals by quantifying the overheads involved by the new design in comparison to the initial one. The obtained results are very encouraging, and allow to extend significantly the potential of the LOCARN use cases. Moreover, the second proposal permits to extend the architecture potential by providing this latter the technical mean to achieve point-to-multipoint transmissions without adding any external multicast protocol.**

***Keywords*—packet network, flat architecture, scalability**

I. Introduction

LOCARN (i.e. Low Opex & Capex Architecture for Resilient Networks) is a very innovative packet architecture that we recently proposed in [1]. It is a flat, dynamic and very simple packet architecture that focuses on plug-and-play guidance to provide flexibility and resiliency on the transport of client data traffics. The big counterparty of the solution is a significant overhead production, which is due to a important generation of control packets over time. In the previous work cite above, we proved that in typical meshed operators transport networks applications, (i.e. infrastructures having high datarates and high resiliency requirements), the LOCARN overheads are acceptable until thousands of communications. Since our previous work finally pointed out that the actual limiting parameter of LOCARN's scalability was the number of communications, we propose in this paper two design evolutions that should permit to increase the amount of communications while maintaining the suitable properties of the initial design. The two proposals constitute what we call a "Large Scale" LOCARN design that should allow the deployment of this solution in much more contexts.

The paper is organized as follows. In section II, we remind the initial LOCARN design and explain the motivations for design improvements. In section III, the principles of our two proposals are presented, and in section IV the overhead results between the initial and the proposed design are compared and explained. Finally we point out some future works in section V before to conclude the paper in section VI.

II. Background & motivations

The LOCARN functional architecture in its initial design has been presented in [1] (reader may refer to the section II that provides a detailed description). In this section, we briefly remind the principles of LOCARN in order to understand the two proposals exposed subsequently.

A. Brief Reminder of LOCARN Mechanisms

The LOCARN functional architecture (see Fig.1) is composed of two kinds of nodes: (i) Edges Nodes (ENs) that constitute ingress/egress nodes of a LOCARN domain (ii) Transit Nodes (TNs) that solely operate packet forwarding[1]. Moreover, the architecture defines: (i) Access Ports (APs) that are external ENs ports (they can be physical or logical ports according to the implementation choice) (ii) Services that are end-to-end bidirectional channels provided for the point-to-point transport of client information between two APs across a domain. A service is composed by two Reference Points (RPs) sharing the same service identifier: a Connection Origin RP (CORP) and a Connection Destination RP (CDRP).
To operate LOCARN, the only management operations to assume are the registration and unregistration of services' Reference Points by declaring unequivocal names among the domain. Then the architecture is in charge of establishment, maintenance and adaptation of services' paths over time through three network mechanisms:

- *autoforwarding (data plane)*: to transmit packets from one point to another (CORP to CDRP), the complete sequence of ports is inserted in the packet header at the ingress EN
- *enhanced flooding (control plane)*: to obtain the port list associated with a service (CORP), an ingress EN performs a source routing mechanism based on network flooding for path discovery[2] (followed by the selection of the best path)
- *end-to-end fault detection (Operation And Maintenance, OAM)*: to improve the architecture resiliency,

[1]"Edge" and "Transit" are convenient functional designations that are in fact relative to the service(s) considered, in practice a node can be Edge and Transit at the same time

[2]the LOCARN flooding propagation is bounded by a Time To Live (TTL) mechanism set in number of hops. The study of the TTL setting impact on the path discovery's overhead have been extensively studied in [1]

978-1-4799-7470-2/14/$31.00 ©2014 IEEE

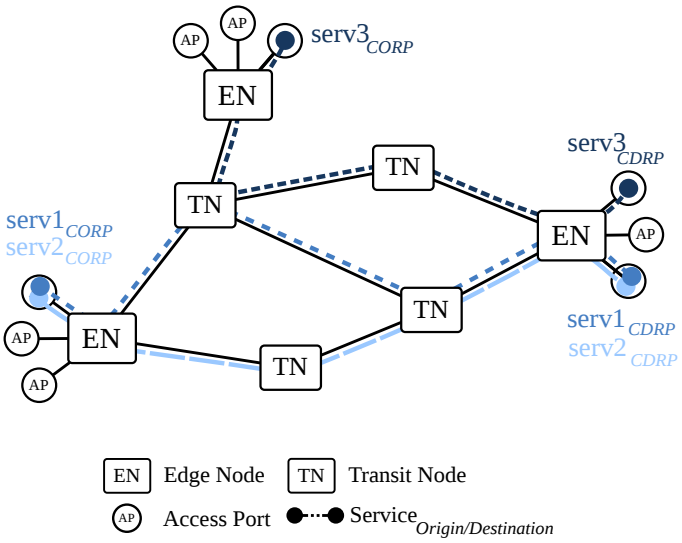


Fig. 1: The LOCARN functional architecture on a small example

small packets are frequently exchanged from end-to-end for each service path in order to detect quickly a path disruption and relaunch a source routing process (which allow a purely reactive path recovery)

The remarkable specificities of the LOCARN source routing are the discovery of numerous potential paths, and the collection of extra-information about each path[3]. Hence, the LOCARN routing is able to qualify each discovered path with a kind of Quality of Service (QoS) attribute, and to determine a best path accordingly – for example in Fig. 1, serv1 and serv2 are able to chose distinct end-to-end paths.
Another important LOCARN's feature consists in the fact that each service re-launches periodically its own routing process. Both path discovery and path selection are performed asynchronously and individually service per service. This makes the global paths distribution adapting over time in accordance with: (i) individual services desired transportation (e.g. favor delay, bandwidth) (ii) the overall network state evolution (e.g. present links, nodes, available bandwidth, queuing states) (iii) the overall evolution of effective clients' traffics.
On the whole, LOCARN can be seen as a network layer in constant adaptation to both the actual infrastructure resources (server layer) and the current traffics fluctuations (client layer). For this reason, it can be qualified as "self-adaptive", "best effort" and "hollistic". Moreover, contrary to other solutions (like suggested by PCE approaches or by many autonomic network architectures), LOCARN solution is simple and relies on a flat architecture, where no entity is involved to coordinate or research an optimal configuration of paths and resources distribution.

B. Motivation of the Two Proposals

In our previous work [1], we made several observations related to LOCARN's performances and scalability.
Concerning the routing plane, we explained that the counterparty of the hollistic paths distribution and adaptations was the amount of overhead involved by periodical floodings. Globally, this overhead depends on: first, the magnitude of one flood (which is a function of the network topology and the flooding Time To Live) and secondly the interval between floods. The impact of those three factors has been extensively studied in [1]. Concerning the floodings overhead minimization, two previous works can be noticed [2], [3]. Related to a close architecture called APLASIA the judicious proposal consists in the reduction the floodings packets generation by a statistical attenuation of their propagation. Yet in LOCARN, a such approach is less beneficial because it reduces the amount of path discovered.
Concerning the OAM, we explained that the ability to recover upon a path disruption in few milliseconds and in a purely reactive approach is also possible to the price of a non negligible overhead cost because of the very frequent end-to-end exchanges.

The LOCARN design presented above is "*service oriented*". Indeed the previous three mechanisms are built around the service functional entity. In particular, source routing and OAM fault detection are performed for each service. This is a notable difference with usual packet-based network standards that consider at least a node level (for routing or path recovery). Based on this observation, we describe in the next section two proposals that aim to de-correlate the control plane and OAM overheads from the number of declared services while maintaing the per service path determination. To do so, we have to introduce some quite more complex mechanisms in both Edge and Transit Nodes.

III. Principles of the Two Proposals

A. First Proposal: Multi-Services Path Discovery

Our first proposal is simple but efficient. In the "service oriented" routing approach of the initial LOCARN design, if S services origins (CORPs) are declared on a same node, S path requests are sent for the services connectivity establishment, but above all S path requests are periodically sent for path optimizations. To reduce the overhead, we simply observe that a single flooding could be launched for the discovery of all paths toward the desired destinations. Instead of launching S flooding requests that looks like "i'm looking for the destination of service A", an Edge Nodes can flood a message looking like "i'am looking for destinations of service A, B, C, ..." and thus collects all answers in one request round.
This Multi-services request approach has two consequences: (i) first, it drastically decorrelates the amount of generated request packets from the number of active services on the domain – which is what we were looking for; (ii) secondly, it increases the size of request packets due to the concatenation of multiple service identifiers. Globally, the correlation between the flooding overhead and the service remains but the number of request packet is drastically reduced when the number of services per Edge Node is important. Finally, concerning the routing behavior, this solution should give at first sight less fineness than the initial "service oriented routing". Thus a trade off should be found between the level of fineness and the overhead gain expected.

In terms of implementation, the request packets (that are flooded across the domain) now carry a list of service identifiers rather than a single one. When a destination node (EN)

[3]typically related to the current state of resources usage like queuing states or recent bandwidth occupation

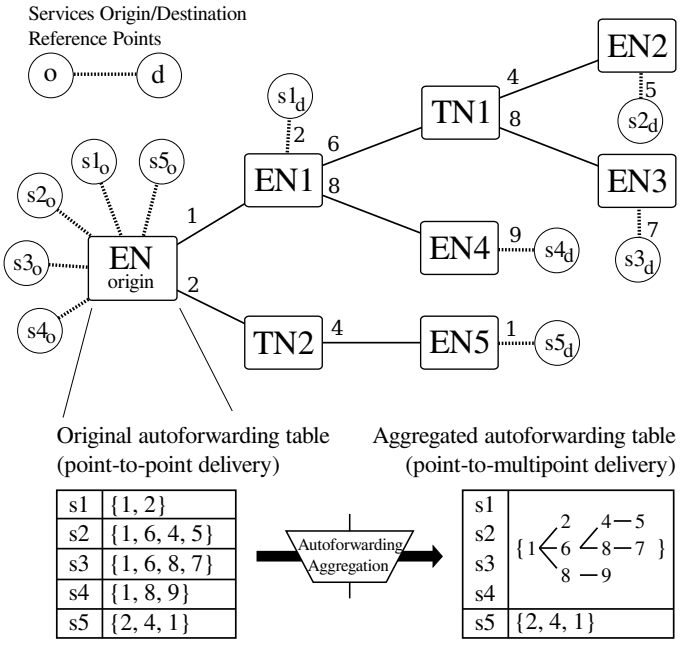


Fig. 2: Illustration of the point-to-multipoint autoforwarding aggregation function on a small example

receives such a packet, it returns a response for each service identifiers from the list which match with a registered service's destination reference point.

B. Second Proposal: Point-to-multipoint Autoforwarding

Our second proposal is quite more complex. As stated before, the initial LOCARN architecture uses "autoforwarding". It means that a packet includes in its header a sequence of ports, allowing each transit node to directly read and switch the packet along its path – a cursor is memorized and incremented at each hop to pointer directly to the overhead part.
In practice, autoforwarding is used in LOCARN both for the data packet and OAM packets transmissions, and constitutes a strong point of the architecture since it is both very simple to implement and very efficient[4]. What we propose here is to extend autoforwarding for point-to-multipoint transmissions. Hereafter, we call point-to-multipoint autoforwarding the ability to transmit a packet starting from one point towards several distinct destinations by using exclusively the information contained in its initial header.
In our architecture the interest is twice: (i) it can be used for data packets transmissions, making the architecture able to transport point-to-multipoint traffics, i.e. to achieve "multicast" ; (ii) it can also be used for grouping the OAM packets emitted by Edge Node. Doing so, the OAM overhead is strongly decorrelated from the amount of services. In order to complement our first proposal (multi-services path discovery), we focus now on the evaluation of the second purpose.

In terms of implementation, the first idea is to use tree data structures instead of lists for the representation of the forwarding ports. The second idea is to build the desired

[4]indeed, TNs have no Forwarding Information Base to store and maintain up to date. As a path is only stored at the EN, no convergence time is involved: a client traffic can be redirected instantly if a better path is found by the routing plane (even frame by frame)

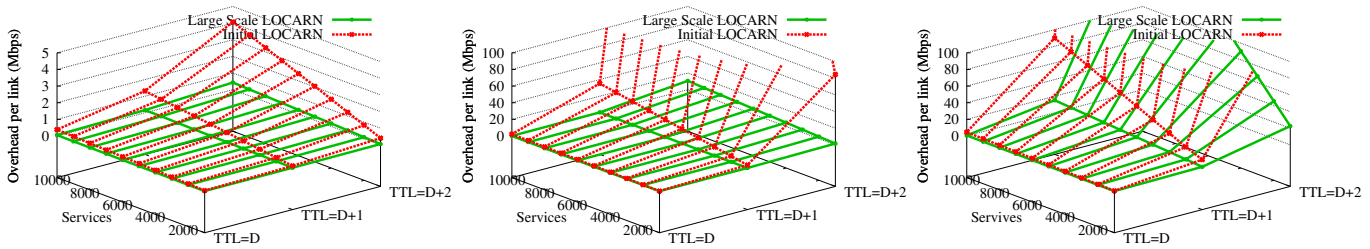
Algorithm 1 Point-to-multipoint Autoforwarding Algorithm

```
1:  procedure AUTOFORWARDINGPROCESSREC(packet p)
2:      header h ← readHeader(p)
3:      integer i ← readFwdHop(h)
4:      integer fwdCode ← readNextFwdCode(h, i)
5:      integer fwdPort ← readNextFwdPort(h i)
6:      if {fwdCode = 0} then
7:          // perform usual point-to-point autoforwarding
8:          setFwdHop(p, i + 1)
9:          sendPacket(p, fwdPort)
10:     else
11:         // fwdCode ∈ [1 − 255] indicates where to bisect
12:         header h1, h2 ← bisectHeader(h, fwdCode)
13:         packet p1 ← assemblePacket(h1, copyPayload(p))
14:         packet p2 ← assemblePacket(h2, copyPayload(p))
15:         sendPacket(p1, fwdPort)
16:         AutoforwardingProcessRec(p2)
17:     end if
18: end procedure
```

autoforwarding trees on the base of the point-to-point autoforwarding information that is already present in LOCARN Edge Nodes. This way, no additional protocol is required for the trees determination (like for example the PIM protocol suite [4]–[6] and IGMP [7] must be added to achieve multicast through an IP network). Two symmetric mechanisms must be defined. In ENs an aggregation function must be introduced for the building of point-to-multipoint tables (P2MP tables). In TNs and ENs, the autoforwarding function must be extended to process adequately point-to-multipoint header structure on the base of a point-to-multipoint algorithm (P2MP forwarding).

As illustrated in Fig.2, the aggregation function is responsible for providing the P2MP tables by using the point-to-point ones. Fig.2 illustrates an example with five services (s1 to s5) terminating on a same Edge Node (called here the "EN origin") and whose destinations are distributed among other ENs. Over time the EN origin routing process fill/update the point-to-point autoforwarding table (on the left). Occasionally, the aggregation function is performed and fills in/updates the P2MP table (on the right) by grouping the autoforwarding ports according to their similarities. In the example, s1 to s4 are grouped together on a same tree root because they all begin their autoforwarding list with the port 1, then s2 and s3 paths belong to the same sub-tree branching on the port 6 etc. For the aggregation of OAM packets that we considered thereafter, we aim to regroup all service paths, so we build one table entry for each distinct first P2P port (which becomes a P2MP tree root). Clearly, the final P2MP autoforwarding headers must be encoded by following a tree representation in accordance with the autoforwarding algorithm that will decode them.

The P2MP autoforwarding function is implemented in Transit Nodes and decodes the header information of an incoming packet. This latter is adequately splited (if needed) into several packets before forwarding (see Algorithm. 1). To do so, we consider the P2MP header encoded as a sequence of *fwdChunks* (2 Bytes) composed of a *fwdPort* (1 Byte) coding the next port as usual, and a *fwdCode* (1 Byte) that indicates how to process the forwarding of the P2MP incoming packet. We respect the following convention: if the *fwdCode* is zero,



(a) GEANT Network (b) DTELECOM network (c) LEVEL 3 Network

Fig. 3: Mean overhead bitrates per link due to floods discoveries: with the initial design besides with the first proposal

TABLE I: Topological Dimensions of the Simulated Networks

Network	N	L	Mean Density	Diameter
GEANT	22	37	3.4	4
Dtelecom	68	353	10.4	3
Level 3	46	268	11.7	4

then the packet is processed as a usual point-to-point packet; on the contrary a *fwdCode* exceeding zero both indicates than the packet must be bisected and indicates where to bisect the P2MP header.
The result of the bisection are two packets having two (sub)parts of the initial header, and of course the same payload (that have been copied). The packet based on the first header subpart is ready to be forwarded upon the interface (indicated by *fwdPort*) whereas the P2MP autoforwarding function is relaunched on the second packet whose header may contain several other subheaders. Finally, the recursive P2MP autoforwarding function will be called N times until $fwdCode = 0$ are encountered, where N corresponds to the n-ary tree branching factor of the incomming packet at this node.

The n-ary tree's form resulting from the aggregation function gives us information about the efficiency of path grouping. To estimate this efficiency, the tree density can be used (i.e. the mean internal nodes degree, also called the mean branching factor). But to measure the non redundancy of link usage, we want to take into account the tree balancing by measuring the number of vertices ($|V|$) divided by the tree height: this give us information about both the tree mean density and about its form.

$$TreeAggregationScore = \frac{|V|}{TreeHeight} \quad (1)$$

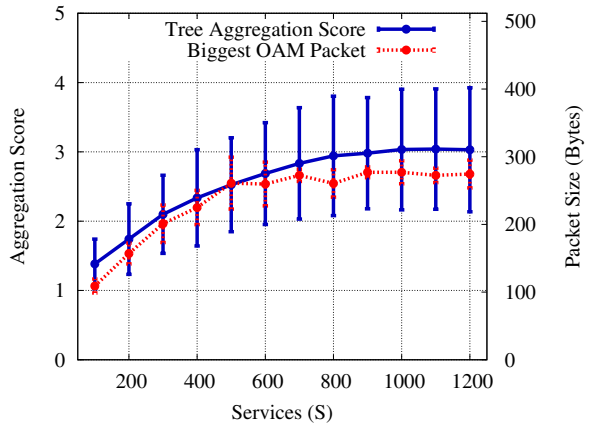
IV. Performances Evaluations

For the LOCARN simulation, we use networks topologies from the CCNSim simulator [8]. The topologies have been included into the LOCARN's simulator, which is built on the *OMNeT++* [9] discret event simulator whereas the n-ary trees structures implementation relies on the *STLplus* [10] C++ library. Thereafter S designates the amount of services declared within a LOCARN domain whereas N and L are respectively the numbers of nodes and links, and δ_{net} designates the network mean density (i.e. the mean number of neighbors per node).

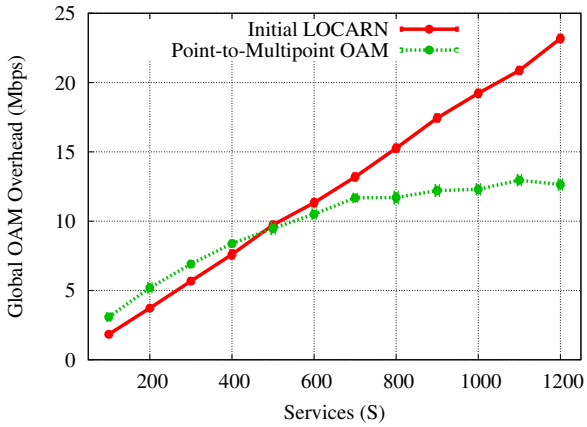
A. First Proposal Evaluation

1) Evaluation Method: Our goal is to estimate the overhead reduction provided by the multi-services path request proposal compared to the initial design. In both designs, the global overhead generation depends on many parameters related to *the network infrastructure*, *the client layer* (i.e. services) and *the* LOCARN*'s settings*. We proceed as follows.
Concerning the network infrastructure, the network topology dimensions (namely the density δ_{net} and diameter D) impact the flooding magnitude[5]. Hence we have selected three well known networks having various topological characteristics (see Table I).
Concerning the client layer, the amount of active services over a domain impacts the overhead linked to the discovery process for both the initial design (each service involves periodical discoveries), and the multi-services based design (packet sizes increase linearly according to the amount of services identifiers to be memorized[6]. Beyond the number of services, the distribution of endpoints within the network is also decisive, what was not the case in the initial design[7]. To make no assumption about the traffic matrix, we distribute the services endpoints randomly within the domain, making the number of service reference points per node tending toward S/N when S is high.
Concerning the LOCARN's parameters, first the flooding packet Time To Live (TTL) which is a key factor of the flooding magnitude (see [1]), is set accordingly to the network diameter: with $TTL = D$ any service is able to find at least one path whereas with $TTL > D$ we allow the discovery of longer paths. Generally, a TTL value of $D + 2$ is sufficient enough to discover a variety of many paths. Finally, the Optimization Interval (OI) which is the duration between two discovery processes also impacts linearly the amount of discoveries launched per time unit. In the two designs, we fixe $OI = 10s$ in order to assure a good LOCARN routing dynamicity.

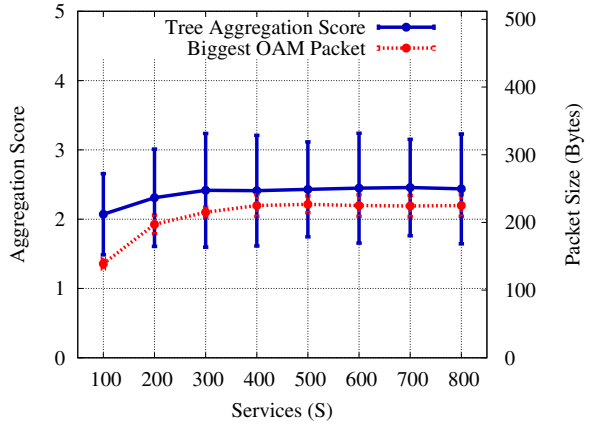
[5] the amount of messages generated by a flooding process
[6] in the multi-services discoveries, since path requests are grouped by nodes, their amount is bound by the number of nodes rather than the number of services
[7] we observed in the three considered networks that the flood magnitude is almost the same whatever the origin node, since the $TTL > D$



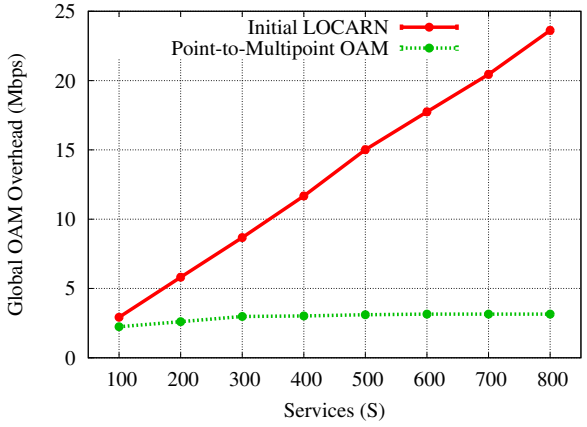
(a) Tree Aggregation for a "Scarse Mode"



(b) Overhead Estimation for a "Scarse Mode"



(c) Tree Aggregation for a "Dense mode"



(d) Overhead Estimation for a "Dense Mode"

Fig. 4: Evaluation of the second proposal on the GEANT Network

To get the results of the Fig. 3, we observe the discovery packets (their amount and their sizes) that have been transmitted during one round of discoveries for all services. By extrapolation, we are then able to give a *mean estimation of the overhead* over time according to the amount of services and the flooding propagation limit (TTL).

2) Interpretation of Results: Fig. 3 exposes as results the mean overheads per link, which permit us to estimate the gain magnitude of the first proposal. Those bitrate values have a very low standard deviation (less than 1%) resulting from data that is quite low (number of packets and packets sizes). Those values permit to compare the overheads without taking into account possible fluctuations over time[8].
What we observe first in all scenarios is that the amount of services has much more impact on the initial design than on the "Large Scale LOCARN" with the first proposal. In Fig. 3a, we can observe that with a network like GEANT (i.e. with a reasonable topology density $\delta_{net} = 3.4$) the interest of our first proposal is globally low (except maybe when both S and TTL are high). In Fig. 3b, we observe that the first proposal is very interesting when S increases and $TTL > D$. In Fig. 3c, we see that with the first proposal the overhead decorrelation from S is important but not total. With very dense topology like LEVEL 3, the flood magnitude is such important that even with our proposal the overhead becomes too important when both S and TTL are high.

[8]can are mostly significant in the initial design because numerous discoveries can occur among very short durations

B. Second Proposal Evaluation

1) Evaluation Method: We now estimate the overhead reduction provided by the use of the point-to-multipoint proposal for the LOCARN OAM's packets transmission. For the two designs, the overhead generated by the OAM packets depends on the number of services and on the service distribution within the network. Hence we assume two modes of services distribution: a "Scarse Mode" where all the reference points (service origin and destination Reference Points) are distributed randomly among the domain; a "Dense Mode" where origins are distributed among an area and destination reference points among another one (areas are composed of contiguous node subsets of about one third of the network).
In Fig. 4, the top (bottom) curves concern the scarse (respectively the dense) service distributions whereas the left and right ones concern the trees aggregation scores and respectively the global OAM overhead comparisons between the initial design and the second proposal. The aggregation scores of left figures

TABLE II: LOCARN's mechanisms and the MPLS typical suite of standards (for future works)

Purpose \ Architecture	LOCARN	Large Scale LOCARN	MPLS
Data Forwarding	Autoforwarding	Autoforwarding	Label Switching (see label stack encoding [11])
Fault Detection	End-to-end detection (point-to-point)	End-to-end detection (point-to-multipoint)	Local detection (links checks, see BFD [12])
Topology Discovery	Periodical floodings (services' individual discoveries)	Periodical floodings (multi-service discoveries)	Link-State diffusion (+TLVs, see ISIS-TE [13])
Path Establishment	N/A	N/A	End-to-end signaling (LSP set up, see RSVP-TE [14])
Path Recovery	Reactive floodings (individual services' recovery)	Reactive floodings (multi-service recoveries)	Local Protection (FRR Extensions to RSVP-TE, see [15])

(solid line, read on left axis) are estimated with the formula (1) averaged among all trees resulting from the aggregations. We also evaluate the maximum sizes of the OAM packet generated (dotted line, read on the right axis), which are related to trees height[9]. The global overheads represented in right figures are obtained by extrapolation of simulation results. After all the services paths have been found, we send one round of OAM packets: for the initial design we simply build packets with the usual point-to-point autoforwarding table and send one packet for each service whereas for the point-to-multipoint version, all Edge Nodes launch their aggregation functions to build their point-to-multipoint autoforwarding tables and then send one "big" packet for each output port. The size of the OAM packets is observed along their end-to-end forwarding. On the base of the cumulated sizes obtained for one round with S services in each design, we estimate the global overheads by considering the Service Check Interval (SCI) equal to 10ms – a such period makes the architecture able to recover service paths under 50ms in most network configurations.

2) Interpretation of Results: As might be expected, the point-to-multipoint OAM design becomes more gainful from a certain amount of services (visible in Fig. 4b) which is due to the introduction of *fwdCode* that is not efficient if the level of aggregation is too low.
Then when S increases, in so far the routing policy leads to spreads the end-to-end paths selected, the level of paths aggregation increases tendentiously. At a certain point, the aggregation score reaches a maximum because no distinct paths are found anymore, trees are somehow "saturated". This makes the gain even more radical because the increase of service has no impact on the packet size anymore. The aggregation saturation is both observable with the *TreeAggregationScore* (Fig. 4a and 4c) and the global overheads (Fig. 4b and 4d). As might be expected, saturation is faster in a Dense than in a Scarse distribution whereas the potential of aggregation is lesser. Finally, we see that on the GEANT network example, the second proposal usage for the OAM overhead minimization would quickly be interesting when the number of services exceeds one thousand.
In terms of performance, the P2MP autoforwarding function based on the Algorithm 1 involves an amount of operation per packet related to the number of tree's branches at the considered step. Hence, the *TreeAggregationScore* values permit us to estimate the number of recursive calls per packet and per node: results are widely acceptable in our example. On the other hand, we can observe that the biggest size for OAM packets, which also depends on the *TreeAggregationScore*, does not become tremendous on our evaluation.

[9]the biggest packets are ones that are sent on the first link and that have not been splited yet

V. Future Works

For future works, the study on LOCARN overheads costs should be compared with some well known packet solutions for backbones networks. We identify MPLS as the closer competitor to LOCARN, typically if this latter involves the following suite of standards (see Table II): the collection of topology information is based on the ISIS-TE routing protocol [13] ; the set up of end-to-end paths (LSPs) is based on the RSVP-TE signaling protocol [14] ; the fault detection is based on the Bidirectional Forwarding Detection (BFD) protocol [12] ; and finally the network protection is based on the Fast Reroute (FRR) mechanism and the corresponding extensions [15]. Overhead comparisons should be done per purpose to be meaningful, and the results should be interpreted in the light of the differences of approaches between LOCARN and MPLS.

Another future work could consists in experimental evaluations of the point-to-multipoint algorithm performances. It would allow to determine the viability of the algorithm for high datarates traffics, as those encountered in backbone networks. Such a study should make vary both the tree aggregation score and traffics datarates.

VI. Conclusion

We have here exposed two proposals for the definition of a Large Scale LOCARN design. Those two proposals permit to reduce drastically the correlation between the number of services and the overhead involved while maintaining the architecture in a service oriented way with regards to path selection and maintenance. Since we pointed out the LOCARN previous works that its scalability determining factor was the amount of services, the two proposals presented and studied here allow to envision the architecture usage in much more contexts, by supporting tens thousands of services even upon large scale network topologies.

References

[1] D. Le Quere, C. Betoule, R. Clavier, Y. Hadjadj-Aoul, A. Ksentini, and G. Thouenon, "Scalability & Performances Evaluation of LOCARN: Low Opex and Capex Architecture for Resilient Networks," in *I4CS 2014. 14th International Conference on Innovations for Community Services*, June 2014.

[2] G. Rossini, D. Rossi, C. Betoule, R. Clavier, and G. Thouenon, "Fib aplasia through probabilistic routing and autoforwarding," *Computer Networks*, vol. 57, no. 14, pp. 2802–2816, 2013.

[3] C. Betoule, T. Bonald, R. Clavier, D. Rossi, G. Rossini, and G. Thouenon, "Adaptive probabilistic flooding for multipath routing," in *New Technologies, Mobility and Security (NTMS), 2012 5th International Conference on*. IEEE, 2012, pp. 1–6.

[4] S. Bhattacharyya, "An Overview of Source-Specific Multicast (SSM)," RFC 3569 (Informational), Internet Engineering Task Force, Jul. 2003.

[5] B. Fenner, M. Handley, H. Holbrook, and I. Kouvelas, "Protocol Independent Multicast - Sparse Mode (PIM-SM): Protocol Specification (Revised)," RFC 4601 (Proposed Standard), Internet Engineering Task Force, Aug. 2006, updated by RFCs 5059, 5796, 6226.

[6] A. Adams, J. Nicholas, and W. Siadak, "Protocol Independent Multicast - Dense Mode (PIM-DM): Protocol Specification (Revised)," RFC 3973 (Experimental), Internet Engineering Task Force, Jan. 2005.

[7] B. Cain, S. Deering, I. Kouvelas, B. Fenner, and A. Thyagarajan, "Internet Group Management Protocol, Version 3," RFC 3376 (Proposed Standard), Internet Engineering Task Force, Oct. 2002, updated by RFC 4604.

[8] "The CCNSIM package is available on the D. Rossi's homepage." [Online]. Available: http://perso.telecom-paristech.fr/~drossi/index.php?n=Software.CcnSim

[9] "The OMNet++ homepage." [Online]. Available: http://www.omnetpp.org/

[10] "The STLplus library's ntrees' documentation webpage."

[11] E. Rosen, D. Tappan, G. Fedorkow, Y. Rekhter, D. Farinacci, T. Li, and A. Conta, "MPLS Label Stack Encoding," RFC 3032 (Proposed Standard), Internet Engineering Task Force, Jan. 2001, updated by RFCs 3443, 4182, 5332, 3270, 5129, 5462, 5586.

[12] D. Katz and D. Ward, "Bidirectional Forwarding Detection (BFD)," RFC 5880 (Proposed Standard), Internet Engineering Task Force, Jun. 2010.

[13] T. Li and H. Smit, "IS-IS Extensions for Traffic Engineering," RFC 5305 (Proposed Standard), Internet Engineering Task Force, Oct. 2008, updated by RFC 5307.

[14] D. Awduche, L. Berger, D. Gan, T. Li, V. Srinivasan, and G. Swallow, "RSVP-TE: Extensions to RSVP for LSP Tunnels," RFC 3209 (Proposed Standard), Internet Engineering Task Force, Dec. 2001, updated by RFCs 3936, 4420, 4874, 5151, 5420, 5711, 6780, 6790.

[15] P. Pan, G. Swallow, and A. Atlas, "Fast Reroute Extensions to RSVP-TE for LSP Tunnels," RFC 4090 (Proposed Standard), Internet Engineering Task Force, May 2005.